

Fast Inference from Transformers via **Speculative Decoding**

arXiv:2211.17192 [[cs.LG](#)]

Published in *40th International Conference on Machine Learning,
ICML 2023*

Presented By: **Havya Sree Karuturi, Kaidi Fu, Martin Chong**



UNIVERSITY OF
ILLINOIS
URBANA-CHAMPAIGN

Presentation Outline

(01) Background & Motivation

(02) Methodology

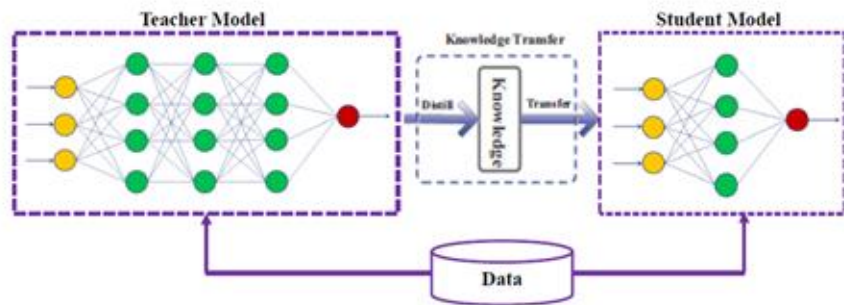
(03) Evaluation

(04) Strengths & Weaknesses

(05) Future Directions

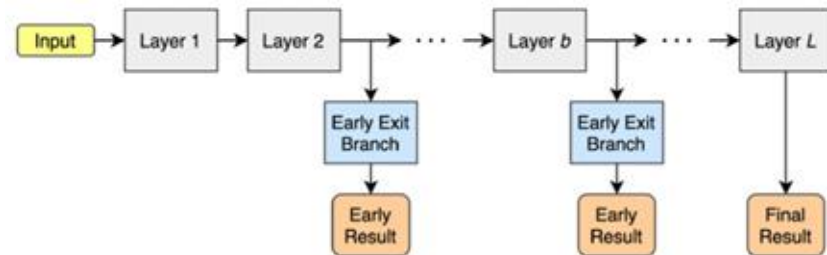


Background



Uniform Reduction

- Reduces compute across all inference requests.
- Knowledge distillation, pruning, etc
- Requires additional training or fine-tuning phases.



Adaptive Computation

- Dynamically adjusts compute per request.
- MoE, early-exit branches.
- Requires changes to the model architecture.

Inference Bottleneck

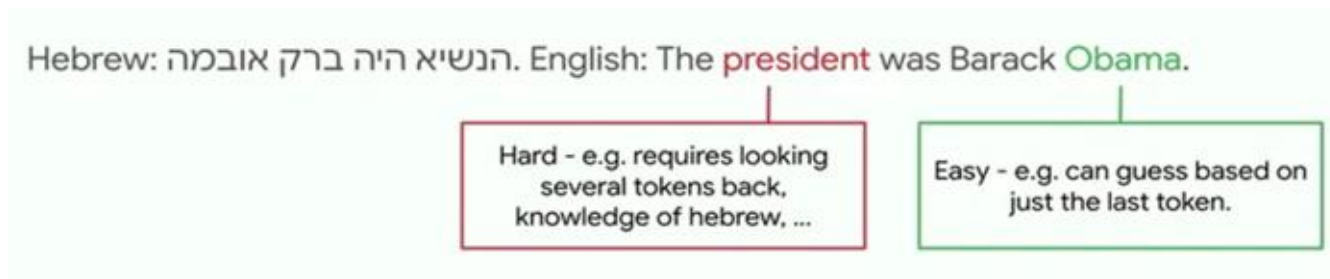
Q: For an autoregressive model, how many **sequential model runs** are required to decode **K tokens**?



Speculative Decoding Motivation

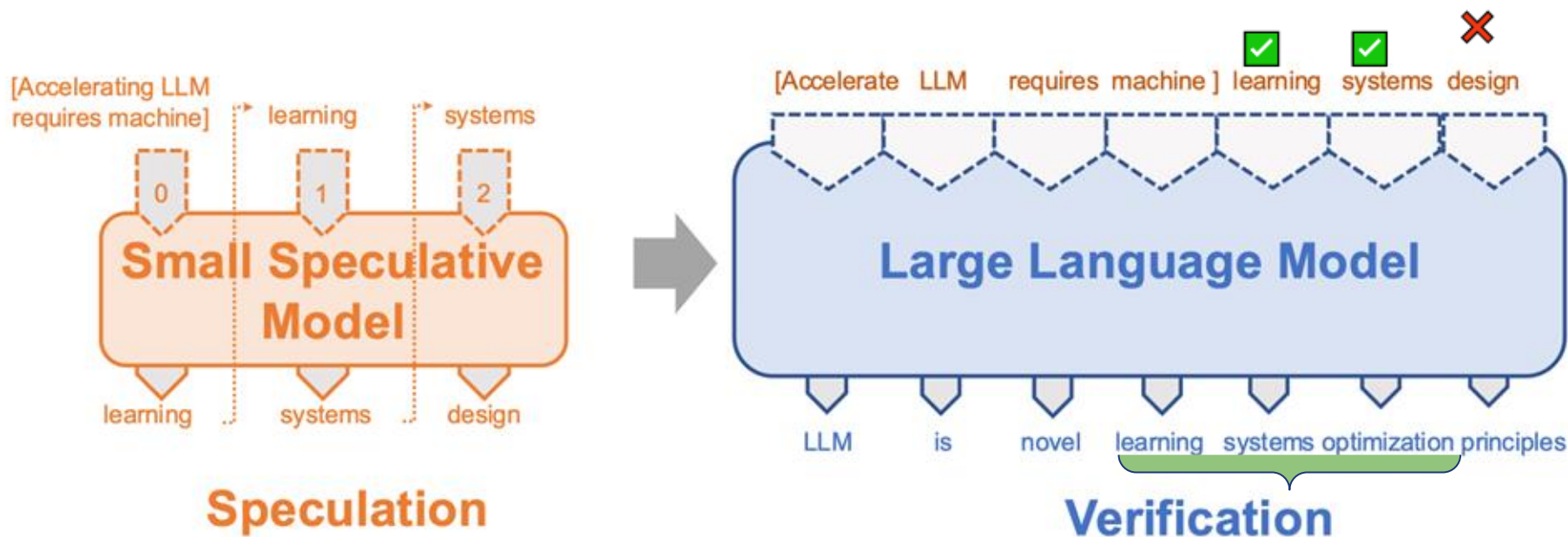
Observation 1: Inference is majorly **memory-bound** rather than compute bound.

Observation 2: Some inference steps are “harder” and some are “**easier**”.

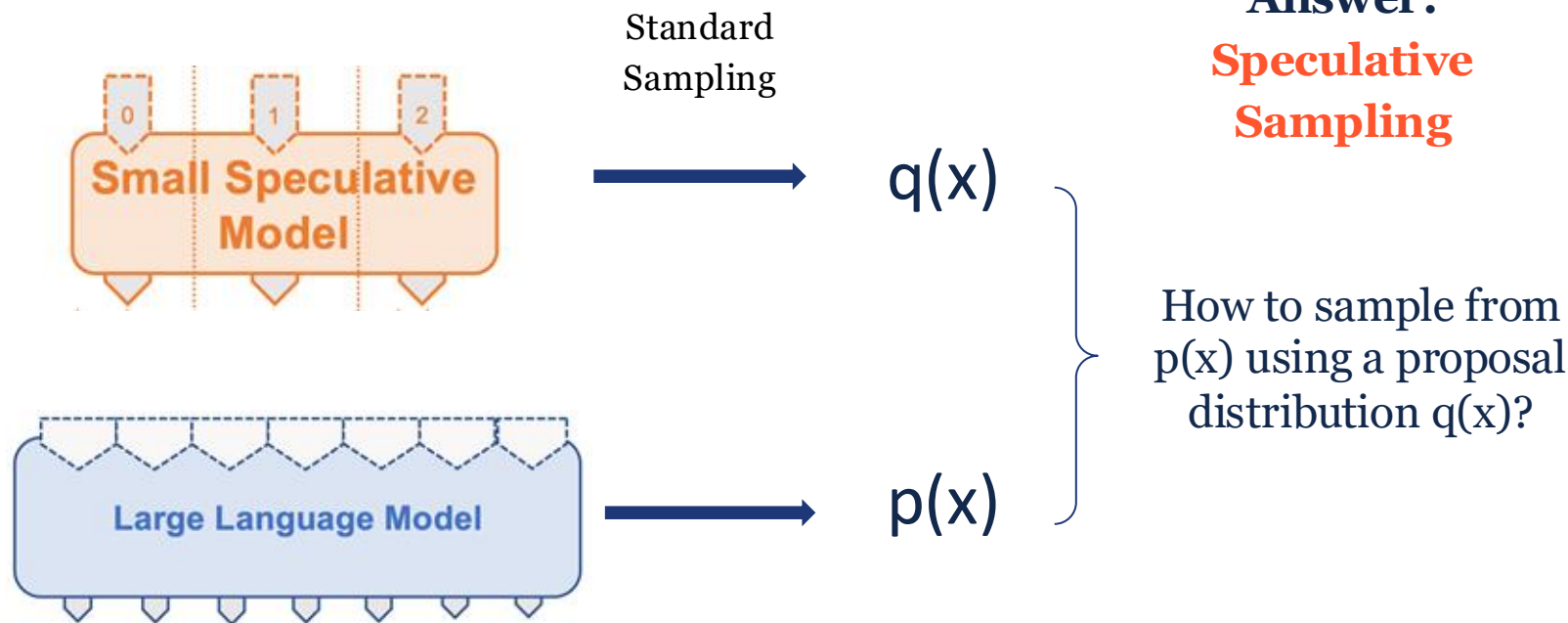


Motivation: Speculative Execution.

Speculative Decoding Overview



What is Speculative Sampling?



Answer:
Speculative
Sampling

Speculative Decoding Process

1 Draft γ guesses

Approximation
model M_q



Autoregressively generate $q_1, q_2, \dots, q_\gamma$ from M_q and sample $x_1, x_2, \dots, x_\gamma$

$$q_i(x) = M_q(\text{prefix} + x_{1:i-1})$$

2 Verify in parallel

Target model M_p



Run M_p once on each drafted prefix: prefix, prefix+ x_1 , ..., prefix+ $x_1:\gamma$.

All target distributions are computed parallelly in a batch.

3 Accept a prefix



If $q_i(x) \leq p_i(x)$:

Accept guess x_i

Else if $q_i(x) > p_i(x)$:

Accept guess x_i with probability $\frac{p_i(x)}{q_i(x)}$

The first failure defines n : keep $x_1, \dots, x_n$ and stop before x_{n+1} , if x_{n+1} is rejected

4 Correct and emit

t

If rejection occurs, sample a correction token from:

$$p'(x) = \text{norm}(\max(0, p(x) - q(x)))$$

If all guesses are accepted, then sample one extra token from $p_{\gamma+1}$



Output: prefix + $[x_1, \dots, x_n, t]$

Correctness of Speculative Sampling

Key idea

Sample from $q(x)$, accept with the right probability, and only resample from the residual mass of $p(x)$.

Proof sketch

Define the acceptance probability $a(x)$, and the resample probability distribution $p'(x)$

$$a(x) = \min\left(1, \frac{p(x)}{q(x)}\right)$$


$$p'(x) = \text{norm}(\max(0, p(x) - q(x)))$$

Combine the two parts of accepting rules to get this

Then we can separate the Probability of finally obtaining $X = x_0$ into two parts

$$\Pr(X = x_0) = \Pr(\text{accepted}, x_0) + \Pr(\text{rejected}, x_0)$$

Correctness of Speculative Sampling (Contd.)

Accepted term:

$$Pr(accepted, x_0) = q(x_0)a(x_0) = \min(q(x_0), p(x_0))$$

**Probability of selecting x_0
from the draft model**

**Probability that target model
accepts the draft token x_0**

Rejected + corrected term

$$Pr(rejected, x_0) = \underbrace{\sum_{x'} (q(x')(1 - a(x')))}_{\text{Probability that the token sampled from draft model is rejected}} * \underbrace{p'(x_0)}_{\text{Probability of resampling } x_0 \text{ from resample distribution } p'(x)} = \sum_{x'} (q(x')(1 - a(x')) * \frac{p(x_0) - \min(q(x_0), p(x_0))}{\sum_{x'} (p(x') - \min(q(x'), p(x'))))}$$

**Probability that the token
sampled from draft model is
rejected**

**Probability of resampling x_0
from resample distribution $p'(x)$**

$$= p(x_0) - \min(q(x_0), p(x_0))$$

$$Pr(X = x_0) = \min(q(x_0), p(x_0)) + p(x_0) - \min(q(x_0), p(x_0)) = p(x_0)$$

Therefore the final token distribution is exactly $p(x)$ for any draft model $q(x)$.

Why It Accelerates

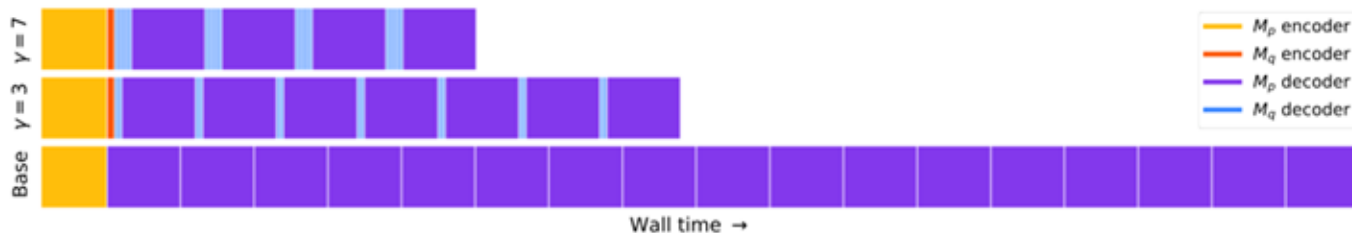


Figure 5. A simplified execution trace: baseline vs. speculative decoding with $\gamma = 3$ and $\gamma = 7$.

Longer γ means one target-model verification pass can validate more draft work in parallel.

Theoretical Speedup

Expected Acceptance rate

Define α as the expected acceptance rate

$$\alpha = \mathbb{E}\left(\sum_{x'} (q(x')(a(x')))\right) \\ = \mathbb{E}(\min(p, q))$$

Cost coefficient

Define one step cost ratio as c

$$c = \frac{\text{time}(M_q \text{ step})}{\text{time}(M_p \text{ step})}$$

Step output

Expected tokens generated by one speculative iteration

$$\mathbb{E}[N] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

Speedup

Expected improvement factor over standard decoding

$$\text{Speedup} = \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(\gamma c + 1)}$$

Choose γ optimally

$$\gamma^* = \arg \max_{\gamma \in \mathbb{Z}_+} \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(\gamma c + 1)}$$

For fixed α and c , search over integer γ and pick the one that maximizes wall-time speedup.

Theoretical Speedup

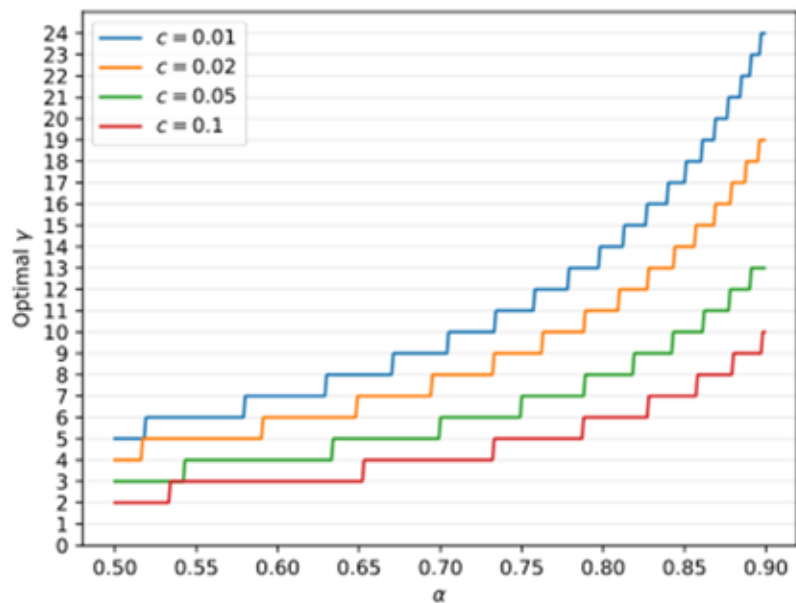


Figure 3. Optimal γ grows as α rises, and shrinks when c is larger.

α is the expected acceptance rate per draft token.
 γ is the fixed number of tokens sampled from draft model each iteration.
 c is the cost ratio between Target model and Draft Model.

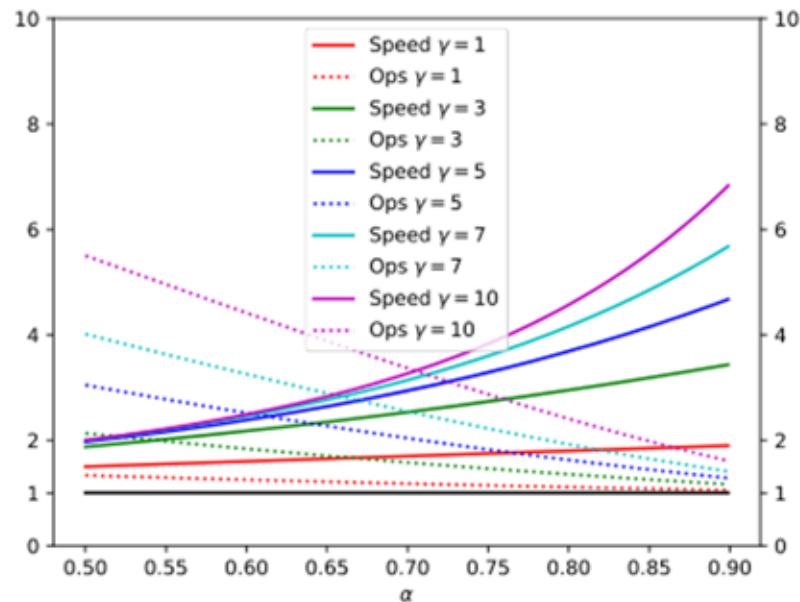


Figure 4. Solid lines: wall-time speedup. Dotted lines: extra arithmetic operations. (Paper assumption: $c = 0$.)

How much walltime speedup is achieved?



Evaluation Setup

Models

- Target-model: T5-XXL (11B parameters)
- Draft Models: T5-Small(77M), T5-Base(250M), T5-Large (800M)

Tasks

- WMT EnDe: English to German Translation
- CNN/DailyMail: News summarization

Experimental setup

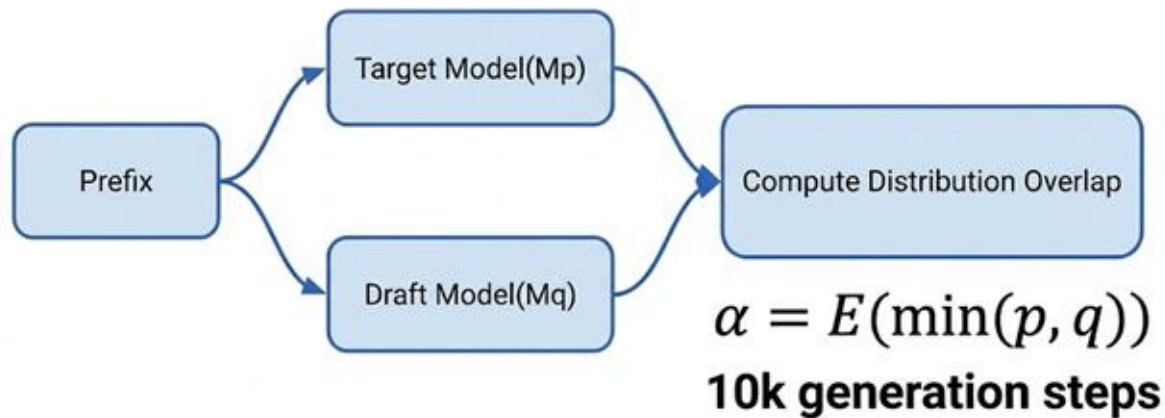
- Batch size : 1
- Hardware: single TPU-v4
- Decoding settings:
 - Temperature 0 (argmax sampling)
 - Temperature 1 (standard sampling)

Walltime Speedup

- Up to **3.4x** speedup in translation
- Up to **3.1x** speedup in summarization
- **Smaller** draft models yield the largest speedups.

TASK	M_q	TEMP	γ	α	SPEED
ENDE	T5-SMALL ★	0	7	0.75	3.4X
ENDE	T5-BASE	0	7	0.8	2.8X
ENDE	T5-LARGE	0	7	0.82	1.7X
ENDE	T5-SMALL ★	1	7	0.62	2.6X
ENDE	T5-BASE	1	5	0.68	2.4X
ENDE	T5-LARGE	1	3	0.71	1.4X
CNNDM	T5-SMALL ★	0	5	0.65	3.1X
CNNDM	T5-BASE	0	5	0.73	3.0X
CNNDM	T5-LARGE	0	3	0.74	2.2X
CNNDM	T5-SMALL ★	1	5	0.53	2.3X
CNNDM	T5-BASE	1	3	0.55	2.2X
CNNDM	T5-LARGE	1	3	0.56	1.7X

How effective are draft models? (Computing α)



How effective are draft models?

- Draft models (~100x smaller) often achieve $\alpha = 0.5 \sim 0.9$
- Even a trivial bigram draft model has $\alpha = 0.2$ and can lead to 1.25x speedup.

M_p	M_q	SMPL	α
T5-XXL (ENDE)	UNIGRAM	T=0	0.08
T5-XXL (ENDE)	BIGRAM	T=0	0.20
T5-XXL (ENDE)	T5-SMALL	T=0	0.75
T5-XXL (ENDE)	T5-BASE	T=0	0.80
T5-XXL (ENDE)	T5-LARGE	T=0	0.82
T5-XXL (ENDE)	UNIGRAM	T=1	0.07
T5-XXL (ENDE)	BIGRAM	T=1	0.19
T5-XXL (ENDE)	T5-SMALL	T=1	0.62
T5-XXL (ENDE)	T5-BASE	T=1	0.68
T5-XXL (ENDE)	T5-LARGE	T=1	0.71

How effective are draft models?

M_p	M_q	SMPL	α
GPT-LIKE (97M)	UNIGRAM	T=0	0.03
GPT-LIKE (97M)	BIGRAM	T=0	0.05
GPT-LIKE (97M)	GPT-LIKE (6M)	T=0	0.88
GPT-LIKE (97M)	UNIGRAM	T=1	0.03
GPT-LIKE (97M)	BIGRAM	T=1	0.05
GPT-LIKE (97M)	GPT-LIKE (6M)	T=1	0.89
LAMDA (137B)	LAMDA (100M)	T=0	0.61
LAMDA (137B)	LAMDA (2B)	T=0	0.71
LAMDA (137B)	LAMDA (8B)	T=0	0.75
LAMDA (137B)	LAMDA (100M)	T=1	0.57
LAMDA (137B)	LAMDA (2B)	T=1	0.71
LAMDA (137B)	LAMDA (8B)	T=1	0.74



Strengths

Identical output distribution

- Most other acceleration methods (early exit, distillation, pruning) change the model's behavior.

Model-agnostic design

- Works with decoder-only models, encoder-decoder models.

Cheap approximation models work well

- Even bigram models could achieve $\alpha = 0.2$ and yield 1.25x speedup.



Limitations

Increased compute cost

- Speeding up decoding at the cost of compute. Not helpful where additional computational resources are not available.

Interaction with batching not explored

- Modern systems use batching methods that increase GPU utilization, which may mean extra compute room for speculative decoding may diminish in batched scenarios.

Limited evaluation scope

- Experiments focused on translation and summarization, rather than conversational workloads.



Future Work

- **Replace off-the-shelf approximation models with custom models.**
- **Hierarchical speculative decoding – approximation model itself can be accelerated by an even faster model.**
- **Instead of a fixed number of draft tokens(γ), use a variable number.**



Future Work

- **Replace off-the-shelf approximation models with custom approximation models.**
- **Hierarchical speculative decoding – approximation model itself can be accelerated by an even faster model.**
- **Instead of a fixed number of guesses(γ), use a variable number.**

More recently...

- **Medusa (ICML 2024) - Adds extra prediction heads to the same model to remove the need for a separate draft model.**
- **EAGLE (ICML 2024) - Accelerates decoding by speculating future hidden states instead of tokens.**



NanoFlow: Towards Optimal Large Language Model Serving Throughput

(Kan Zhu, et al.)

arXiv:2408.12757

Published in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 2025.

Presented By: Michelle Adeline and Kaizhuo Chen



UNIVERSITY OF
ILLINOIS
URBANA-CHAMPAIGN

Compute vs Memory vs Network Bound Operations

Q: Is **Decoding Attention** memory
or compute bound?

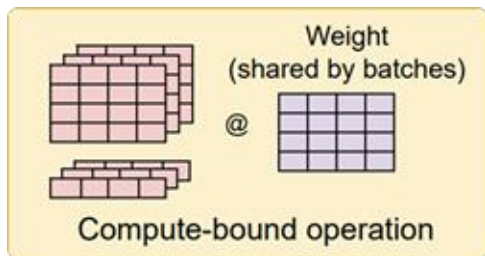
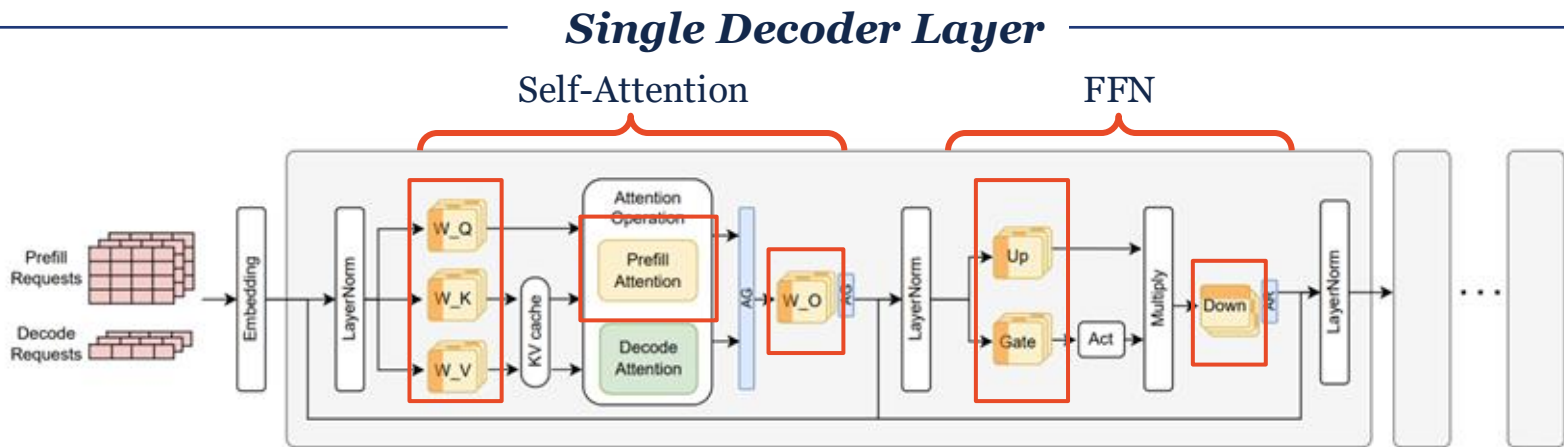


Compute vs Memory vs Network Bound Operations

Q: Is **FFN** memory or compute bound?



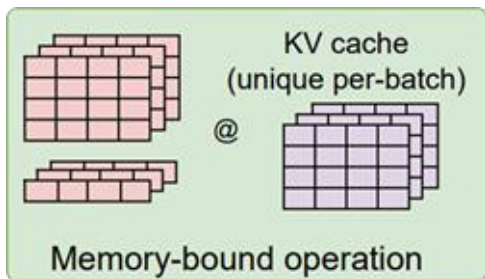
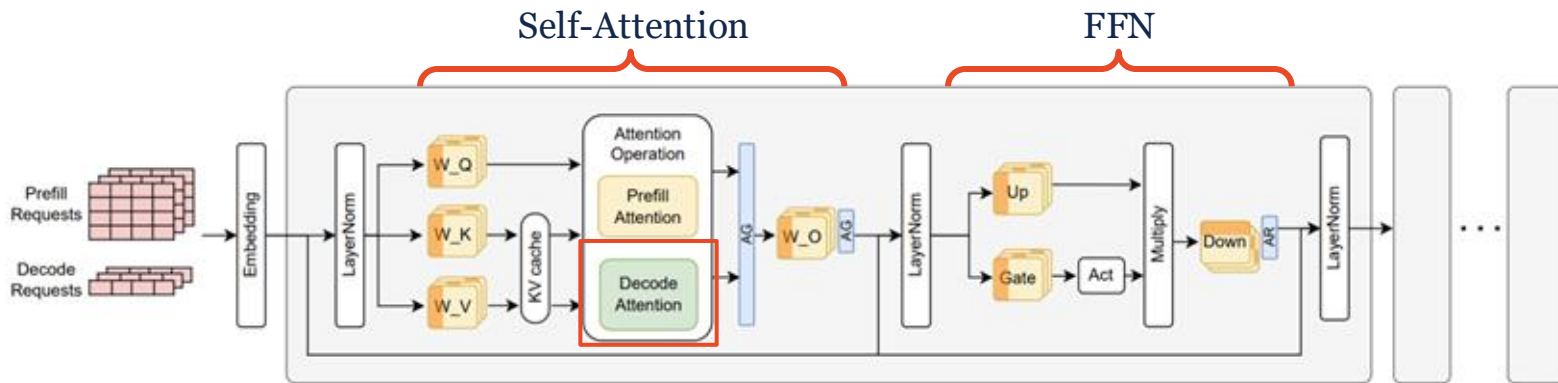
Compute vs Memory vs Network Bound Operations



- KQV generation, O Projection, Up, Gate, and Down Projection all involves matrix multiplication of input with model weights.
- If we batch inputs, weights are shared by the entire batch which **amortizes weight loading cost**, making them compute-bound.
- Prefill attention processes all tokens at once, so is also compute-bound.

Compute vs Memory vs Network Bound Operations

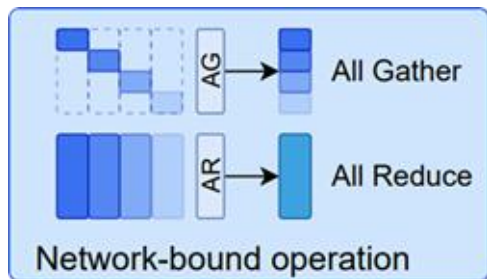
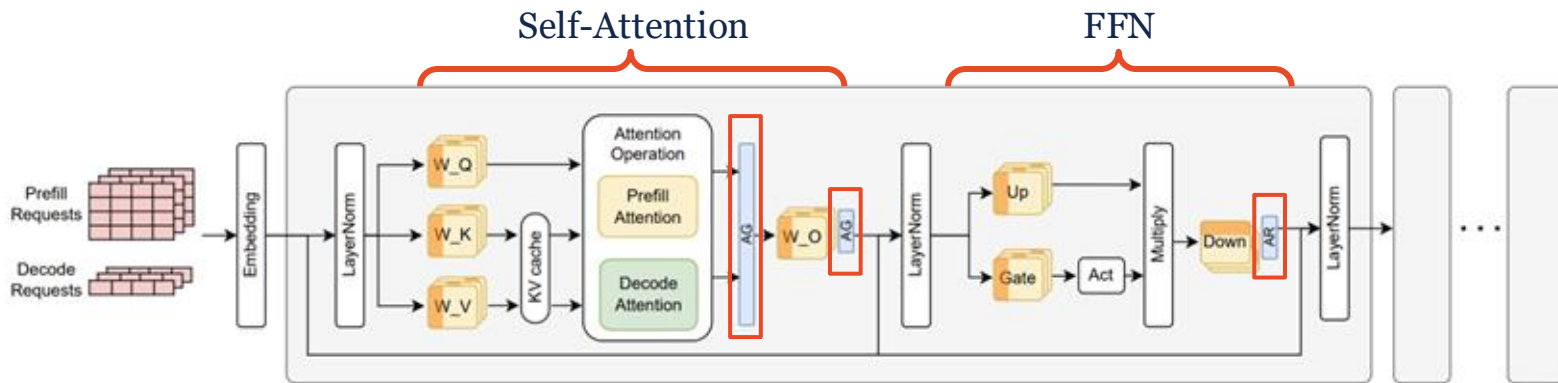
Single Decoder Layer



- Decode attention needs to load **KV cache** which is **unique per request** AND only generates one token at a time, making it **memory-bound**.

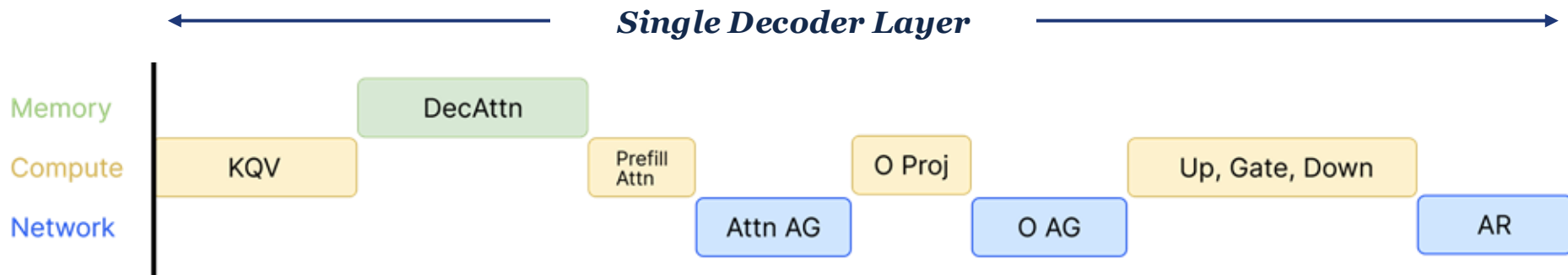
Compute vs Memory vs Network Bound Operations

Single Decoder Layer



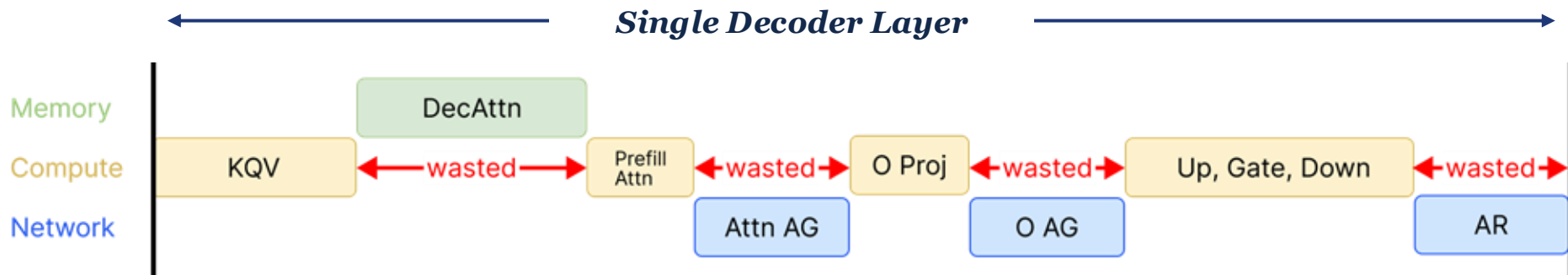
- When Tensor Parallelism is applied, All Gather and All Reduce are needed to synchronize inputs/outputs between GPUs.
 - For ex. Each attention head is executed on a separate GPU
- These operations are **network-bound**.

Existing Systems



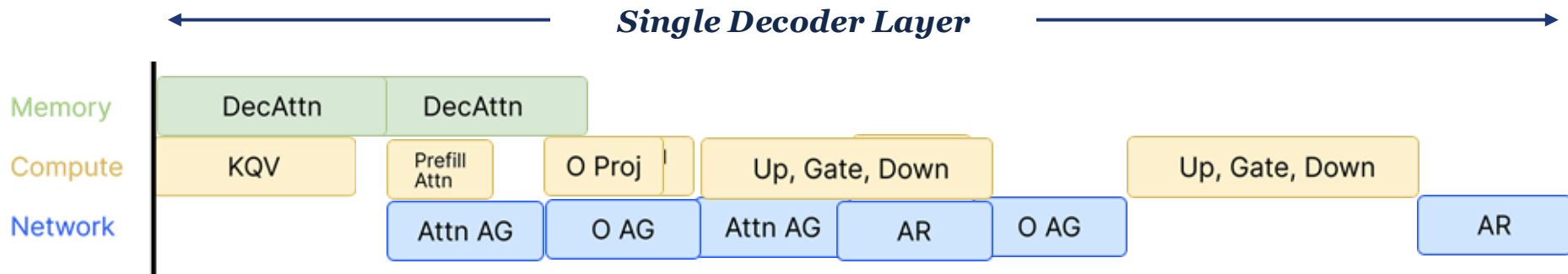
- Existing systems (for ex. vLLM, TensorRT-LLM) execute these operations sequentially

Existing Systems



- Existing systems (for ex. vLLM, TensorRT-LLM) execute these operations sequentially
- Since different operations are constrained by different resources, this creates wasted resources
 - For ex. During decoding attention, which is memory-bound, say that it uses 100% of GPU memory but only 60% of GPU compute. **The remaining 40% of compute is idle!**

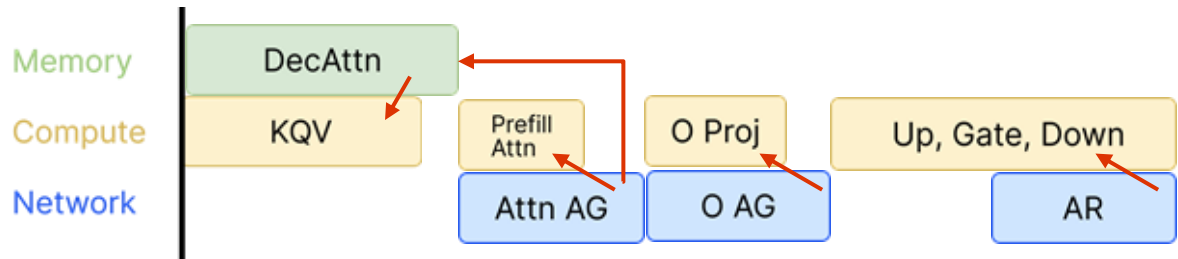
Overlapping Heterogeneous Resources



- The core idea of NanoFlow is to overlap operations that are constrained by different resources to maximize GPU utilization

Overlapping Heterogeneous Resources

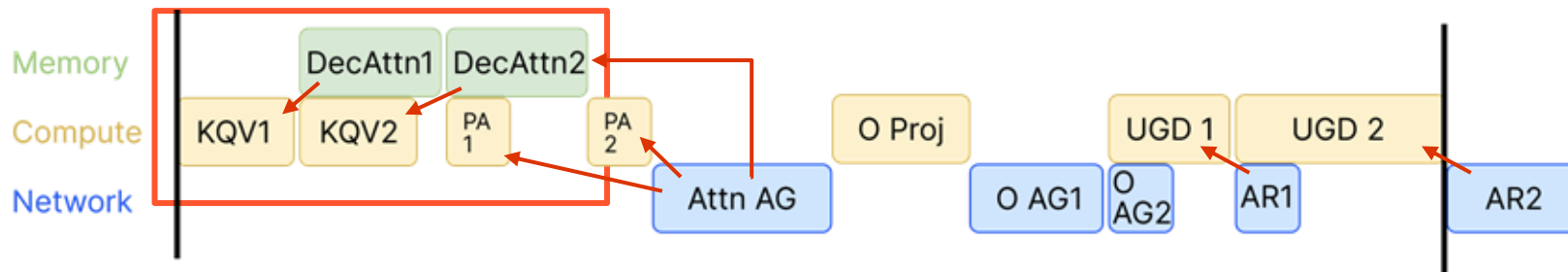
Issue: Dependencies



- But this is not possible due to **dependencies**, for ex:
 - You cannot perform attention without generating KQV first
 - You cannot perform All-Gather across all attention heads when the attention operations have not finished

Overlapping Heterogeneous Resources

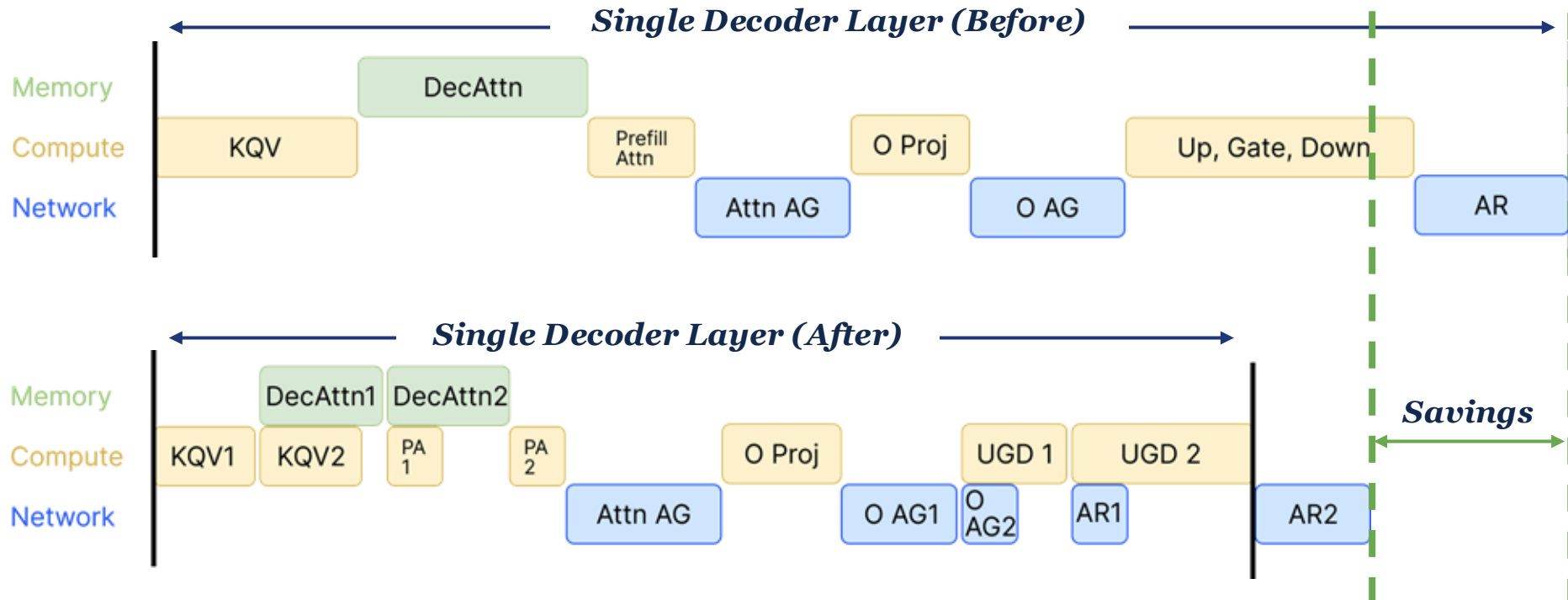
Solution: Nano-Operations



- To resolve this, NanoFlow breaks down operations into **nano-operations** which are run on nano-batches. For ex.
 - DecAttn1 processes tokens 1-200, DecAttn2 processes tokens 200-500
 - DecAttn1 depends on KQV1 but has no dependency with KQV2 so both can be ran at the same time!

Overlapping Heterogeneous Resources

Solution: Nano-Operations



Overlapping Heterogeneous Resources

Issue: Many Hyperparameters

- 1) Which operations to break into nano-operations?
- 2) How many nano-operations to break into?
- 3) Nano-batch size
- 4) Order of nano-operations

Auto-Search Pipeline Phase I

Inputs:

- 1) Execution Time Map
- 2) Dependency DAG

Mixed Integer Linear
Programming



Goal: Minimize
Compute Bubbles

Outputs:

- 1) Which operations to break into nano-operations?
- 2) How many nano-operations to break into?
- 3) Nano-batch size
- 4) Order of nano-operations



Auto-Search Pipeline Phase I

Inputs:

- 1) Execution Time Map
- 2) Dependency DAG

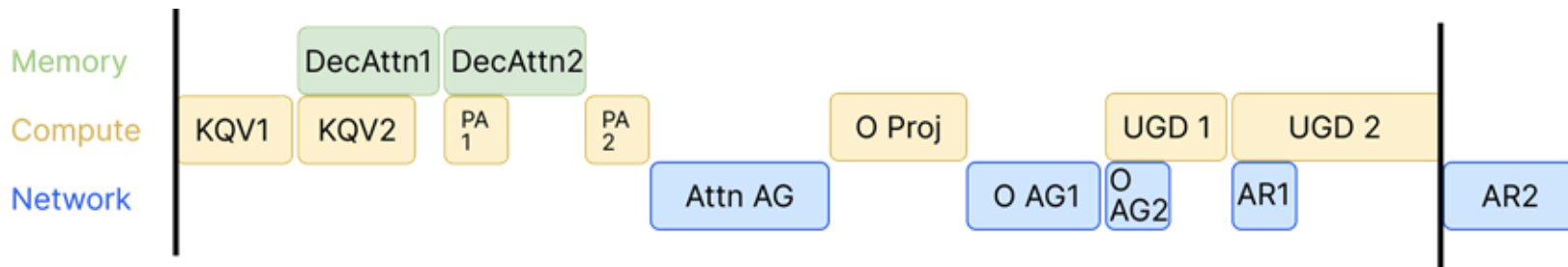
Mixed Integer Linear
Programming

Goal: Minimize
Compute Bubbles

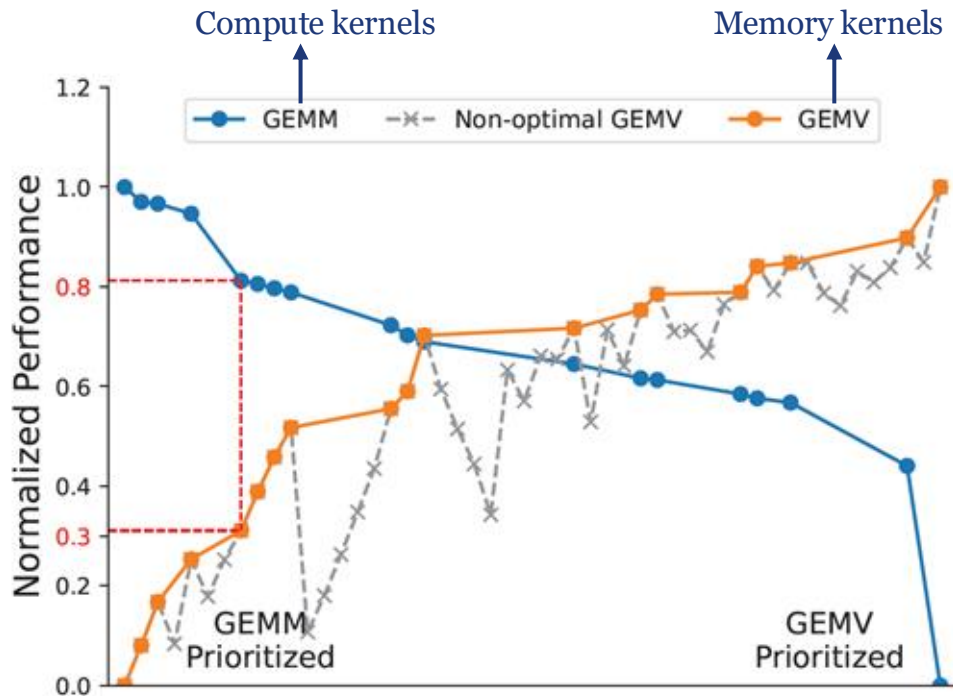
Outputs:

- 1) Which operations to break into nano-operations?
- 2) How many nano-operations to break into?
- 3) Nano-batch size
- 4) Order of nano-operations

Phase I Output



Interference



- Unfortunately, when two GPU kernels run at the same time, they **interfere with each other, preventing each kernel from reaching 100% performance.**
 - For **one (GEMM, GEMV) kernel pair**, if both are ran at the same time, the GEMM kernel achieves 80% of its peak performance (executed by itself) and the GEMV kernel achieves only 30% of its peak performance.
- This interference is kernel-dependent and often non-linear.

Auto-Search Pipeline Phase II

Inputs:

- 1) Execution Time Map
- 2) Dependency DAG
- 3) Hyperparams from Phase I

Mixed Integer Linear
Programming

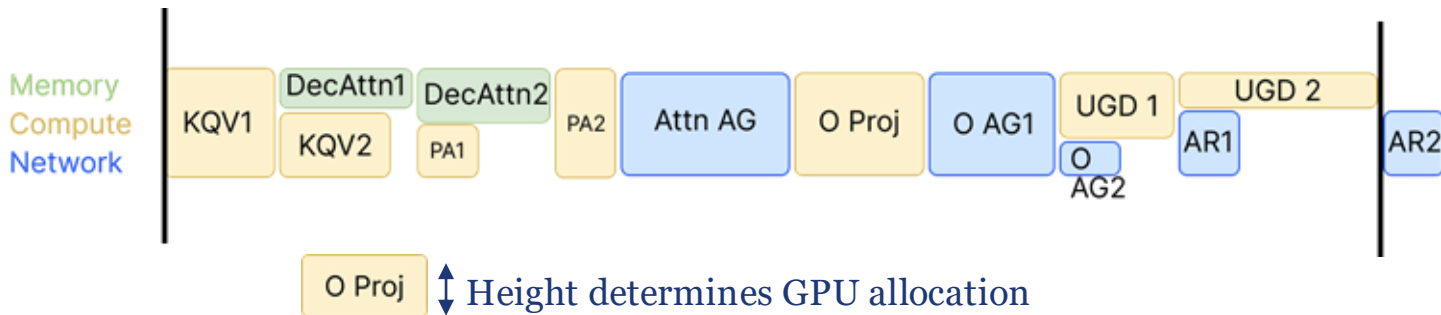


Goal: Minimize
Compute Bubbles

Output:

Resource Allocation for
each nano-operation

Phase II Output



Evaluation: Experiments Setup

Models Evaluated

- LLaMA-2-70B
- LLaMA-3-8B
- Mixtral-8×7B
- Qwen-72B

Serving Systems

- vLLM
- DeepSpeed-FastGen
- TensorRT-LLM

Hardware

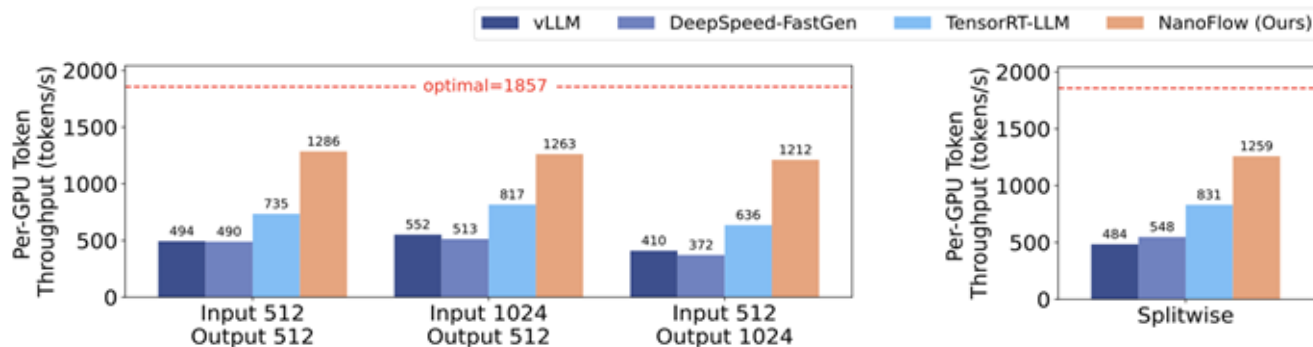
- $8 \times$ NVIDIA A100 80GB SXM GPU

LLaMA-3-8B evaluated on $1 \times$ A100 GPU

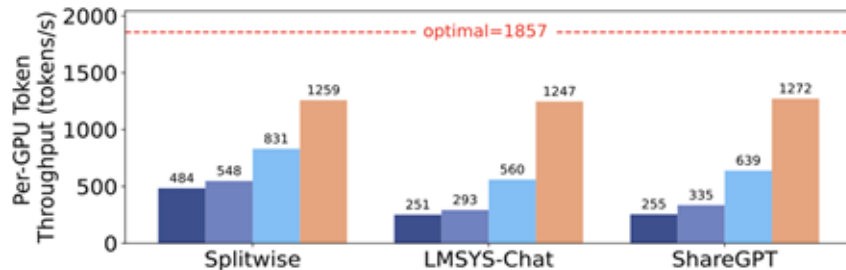
Metrics

- Throughput (tokens/sec)
- Normalized latency (ms/token)

Evaluation: Throughput Results



(a) LLaMA-2-70B, 8 GPU, TP=8, Constant Input & Output Length

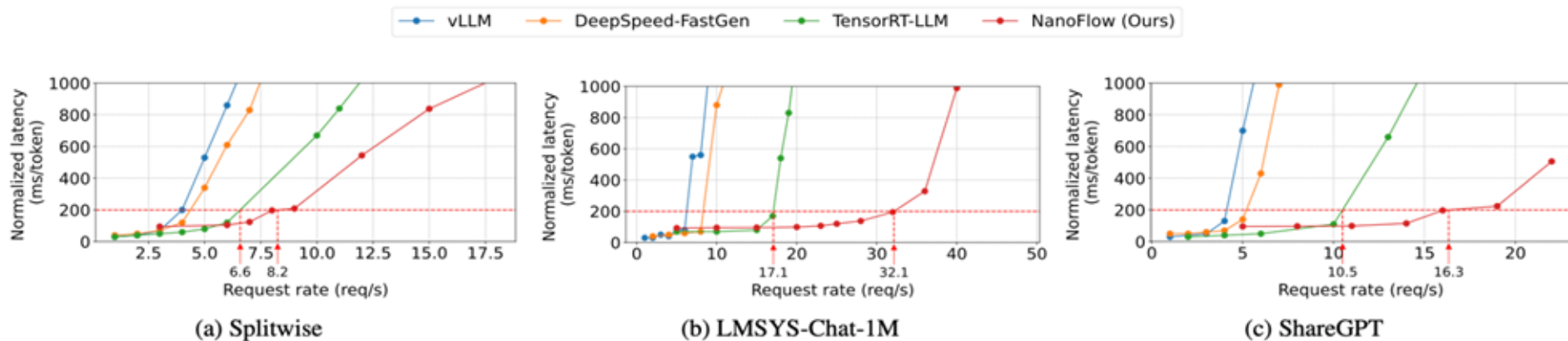


(b) LLaMA-2-70B, 8 GPU, TP=8, Input & Output Length from Dataset

- NanoFlow achieves the highest throughput in all settings
- Up to **68.5%** of theoretical optimal throughput
- **2.62×** throughput compared to **vLLM**
- **2.78×** throughput compared to **DeepSpeed-FastGen**
- **1.73×** throughput compared to **TensorRT-LLM**



Evaluation: Latency Results



- NanoFlow has **slightly higher latency at low request rates**, due to large **dense batch size**
- NanoFlow sustains **higher request rates** under the **200 ms SLO**, **1.64× higher request rate** than TensorRT-LLM on LMSYS-Chat





Strengths

Key Insight: LLM serving is mostly compute-bound

- Most papers (even SpeculativeDecoding) assumes that LLM inference is memory-bound
- Motivates completely different optimization directions for future work

Auto-Pipeline Search

- Avoids manual tuning
- Adapts to different models and hardware
- Fast (~40 mins total)

Strong empirical results

- Convincingly outperforms existing serving systems (~2.6x higher throughput compared to vLLM)



Limitations

Compute-bound analysis has strong assumptions

- Assumes most decode sequences are not long (becomes memory-bound)
- Assumes Grouped Query Attention (GQA)
- Assumes large models + large batch sizes

Simplified kernel interference profiling

- Considers pairwise (**compute**, **memory**) and (**compute**, **network**) kernels only.
- No profiled interference with (**compute**, **memory**, **network**) operations even though their example pipeline has all three occurring at the same time.

Evaluation only on single node



Future Works

Intra-GPU Spatial Multiplexing (Hardware Partitioning)

- NanoFlow relied on temporal overlapping using CUDA streams, which still caused latency spikes due to synchronization barriers between tasks.
- Newer frameworks (like *MuxWise*) **physically partition the GPU's Streaming Multiprocessors** (SMs), so compute-bound and memory-bound phases can execute completely independently.

Lock-Free Dynamic Orchestration

- Systems like *RAPID-Serve* implement **lock-free, asynchronous scheduling**.
- This ensures that massive matrix multiplications from new inputs don't block the generation of current output tokens, minimizing wait times and preventing pipeline stalls.



Thank You



UNIVERSITY OF
ILLINOIS
URBANA-CHAMPAIGN